CPSY 1291 · Recitation handout 8

Comparing representations

RSA, decoding, and how to get an honest p-value

Fall 2026 · Week 8 · hold this after Lecture 12; Assignment 3 is due Thu 11/5

What this session is for

Lecture 12 gives you two ways to put a model next to a brain: compare their representational *geometry* (RSA), or ask what can be *read out* of each (decoding). This session is the machinery — the code, and the three statistical traps that make published versions of these analyses wrong.

One of those traps is a debt from Recitation 3. There we said that correlating two dissimilarity matrices gives a meaningless p-value, because the pairs are not independent, and that the fix was a permutation test. Here is the fix.

> **The one idea.** A similarity score between a model and a brain is uninterpretable on its own. It only means something against two other numbers: what score *chance* would produce, and what score a *perfect* model could produce given the noise in the data. Compute both, always, and report all three.

1 RSA, end to end

1.1 The pipeline

```
from scipy.spatial.distance import pdist, squareform
from scipy.stats import spearmanr

def rdm(X, metric='correlation'):
   """X: (n_stimuli, n_units) -> (n, n) dissimilarity matrix."""
   return squareform(pdist(X, metric=metric))

def rsa(D1, D2):
   """Spearman correlation between two RDMs, upper triangle only."""
   iu = np.triu_indices(D1.shape[0], k=1)
   return spearmanr(D1[iu], D2[iu]).statistic
```

Four decisions are buried in those six lines, and each of them changes the answer:

1. **Which distance.** Correlation distance is the field's default for neural data because it removes each stimulus's overall response level, which is often dominated by attention or arousal rather than by stimulus identity. Euclidean keeps it. Say which you used.
2. **Whether to standardize the units first.** A voxel or unit with a large dynamic range dominates a Euclidean RDM. Standardizing gives every unit equal say. Neither choice is

neutral.

3. **Spearman or Pearson** for comparing the two RDMs. Spearman, for the reason in Recitation 3: the two systems' distances are on unrelated scales, and the question is whether they order the pairs alike.
4. **The stimulus set.** Everything below depends on it, and it is the thing nobody varies.

1.2 The stimulus set is the experiment

An RSA score is a statement about the stimuli you happened to use. If your stimulus set is 40 animals and 40 tools, almost any representation that distinguishes animate from inanimate will score well, and the score is telling you about your stimulus set rather than about the model. Two models that disagree everywhere can look identical on a set with one dominant division.

> **The question to ask of any RSA result.** What would this score be for a model that only knew the coarsest category distinction in the stimulus set? If the answer is "about the same", the analysis has not distinguished anything.

2 CKA, in five lines

Lecture 6 introduced CKA alongside RSA, and Lecture 7 showed that changing the measure can change which model wins; CKA is the current default alternative to RSA and is worth having in code, because comparing two metrics on the same data is a good way to see how much a conclusion depends on the metric.

```
def cka(X, Y):
    """Linear CKA between two (n_stimuli, n_units) matrices."""
    X = X - X.mean(0)
    Y = Y - Y.mean(0)
    num = np.linalg.norm(X.T @ Y, 'fro') ** 2
    return num / (np.linalg.norm(X.T @ X, 'fro') * np.linalg.norm(Y.T @ Y, 'fro'))
```

CKA is invariant to rotations and to isotropic scaling but *not* to arbitrary invertible linear maps, which is what makes it more discriminating than CCA. Both centerings are per-unit (column), and forgetting them is the usual bug — it inflates the score toward 1 for almost any pair of matrices.

3 Permutation tests: an honest p-value

3.1 The logic

A p-value answers: *if there were no relationship, how often would I see a score at least this large by chance?* When you cannot appeal to a formula — because your data violates its assumptions — you can answer the question directly, by destroying the relationship many times and looking at what scores survive.

```
def rsa_permutation_p(D1, D2, n_perm=10_000, rng=np.random.default_rng(0)):
   n = D1.shape[0]
   obs = rsa(D1, D2)
   null = np.empty(n_perm)
   for i in range(n_perm):
      p = rng.permutation(n) # shuffle STIMULI, not pairs
      null[i] = rsa(D1, D2[np.ix_(p, p)]) # rows and columns together
   return obs, (np.sum(null >= obs) + 1) / (n_perm + 1)
```

> **Two details that are the whole test.** **Shuffle stimuli, not pairs.** Permuting the $n(n-1)/2$ entries of the upper triangle destroys the matrix structure as well as the relationship, and gives a null distribution far too narrow — so everything comes out significant. Permuting stimulus labels and reindexing both the rows *and* the columns (`np.ix_(p, p)`) keeps a valid dissimilarity matrix and destroys only the correspondence. **And add one** to both numerator and denominator: with 10,000 permutations the smallest honest p is $1/10001$, not 0.

3.2 Reading the null distribution

Plot it. A histogram of the 10,000 null scores with your observed value marked is more informative than the p-value it produces, and it catches the failure mode above: a null distribution concentrated in a spike near zero means you permuted the wrong thing.

3.3 Bootstrap: a confidence interval on the score itself

The permutation test asks whether the score differs from chance. A different and often more useful question is how precise the score is — which is answered by resampling stimuli with replacement:

```
boot = []
for _ in range(1000):
   idx = rng.integers(0, n, n) # with replacement
   boot.append(rsa(D1[np.ix_(idx, idx)], D2[np.ix_(idx, idx)]))
lo, hi = np.percentile(boot, [2.5, 97.5])
```

This is what you need to say "model A fits better than model B": compare their intervals, or better, bootstrap the *difference* between them.

4 The noise ceiling

Neural data is noisy. Split your subjects (or your repeats) in half and correlate one half's RDM with the other's: that number is roughly the best any model could achieve, because no model can predict the noise.

```
half1 = rdm(X[subjects_a].mean(0))
half2 = rdm(X[subjects_b].mean(0))
```

```
ceiling = rsa(half1, half2) # average over many random splits
```

> **Why this changes every conclusion.** A model scoring 0.35 against a ceiling of 0.40 is doing very well. The same 0.35 against a ceiling of 0.85 is doing poorly. Reporting the raw score without the ceiling makes those two indistinguishable, and it is the reason model rankings in this literature sometimes reverse when the ceiling is added.

5 Decoding

5.1 A decoder is a classifier

```
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score, StratifiedKFold

clf = make_pipeline(StandardScaler(), LinearSVC(C=1.0))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(clf, X, y, cv=cv)
```

The `make_pipeline` is not stylistic. Standardizing outside the cross-validation fits the scaler on data the classifier is about to be tested on — the leak from Recitation 6, in its most common disguise.

5.2 Three ways a decoding result is wrong

Chance is not always 50%. With k balanced classes it is $1/k$. With *imbalanced* classes, a classifier that always answers "the majority class" scores the majority's proportion — 90% accuracy on a 90/10 split is exactly nothing. Report balanced accuracy, or the confusion matrix, or both.

Trial structure leaks. If two examples come from the same trial, the same run, or the same block, splitting them across train and test lets the classifier recognize the trial rather than the condition. Group-aware splitting (`GroupKFold`) is the fix, and its absence is a common and invisible error in neuroimaging.

Decodable is not used. Information being linearly readable from a population does not mean the organism reads it. This is the interpretive limit of the whole method and it is worth stating in any write-up that uses it.

5.3 What C does

`LinearSVC(C=...)` trades margin against training errors: small C means a wide margin and more tolerance of mistakes (more regularization), large C means fitting the training data hard. With more features than examples — the normal case for neural data — this matters

a great deal, and C should be chosen on validation folds nested inside the cross-validation, never on the test fold.

6 Exercises

1. You permute the entries of an RDM's upper triangle and get $p < 0.0001$ for every model you test, including a random one. What went wrong?

2. Model A scores 0.31 and model B scores 0.29 on the same data. What two additional numbers do you need before saying A is better?

3. Your noise ceiling comes out at 0.95. What does that tell you about your data, and is it good news?

4. A classifier decodes stimulus category from V1 at 78% where chance is 50%. Your advisor asks whether V1 "represents category". Give the honest answer in two sentences.

5. You standardize your features, then run `cross_val_score`. Your accuracy is 3 points higher than a colleague's on the same data. Explain.

6. Two models have RSA scores of 0.62 and 0.61 on a stimulus set of 40 animals and 40 tools. What single check would you run before believing the models are equivalent?

Answers

1. You permuted pairs rather than stimuli. Shuffling the upper triangle destroys the matrix structure as well as the correspondence, producing a null distribution far too narrow, so every observed score looks extreme. Permute stimulus labels and reindex rows and columns together.

2. The noise ceiling (is 0.02 large relative to the achievable range?) and a confidence interval on the *difference*, from bootstrapping stimuli. Without those, 0.31 vs 0.29 is not a comparison.

3. It says your measurements are very reliable — probably many repeats, or averaged over many subjects. It is good news for precision, and it also means a model scoring 0.4 is genuinely explaining less than half of what is there, rather than being limited by noise. A high ceiling makes models look worse, honestly.

4. Category information is linearly readable from V1's population response under these conditions. That does not establish that V1 represents category — low-level features that correlate with category in this stimulus set would produce the same result, and nothing here shows the rest of the brain uses this information.

5. You standardized before splitting, so the scaler was fitted using the test folds' means and standard deviations. That is leakage; the colleague's lower number is the correct one. Put the scaler inside a `Pipeline`.

6. Check what a model that *only* knows animate vs inanimate scores. If it also scores about 0.6, then the stimulus set's dominant division is carrying both results and the two models have not been distinguished at all.

Cheat sheet

You want	You write
an RDM	`squareform(pdist(X, 'correlation'))`
an RSA score	`spearmanr(D1[iu], D2[iu]),  iu = triu_indices(n,1)`
permute an RDM's stimuli	`D[np.ix_(p, p)]`
an honest p	`(np.sum(null >= obs) + 1) / (n_perm + 1)`
a CI on a score	resample stimuli with replacement, take percentiles
a noise ceiling	split-half RDM correlation, averaged over splits
a decoder, done right	`make_pipeline(StandardScaler(), LinearSVC())`
folds that respect trials	`GroupKFold(...).split(X, y, groups)`
chance, with imbalance	`balanced_accuracy_score`, or the confusion matrix
linear CKA	`norm(Xc.T@Yc)**2 / (norm(Xc.T@Xc)*norm(Yc.T@Yc))`